Package 'golemworldbank'

October 27, 2026

Title World Bank Indicators Explorer

Version 0.0.1

Author Dimitrios Zacharatos [aut, cre]

Maintainer Dimitrios Zacharatos <dzacharatos@yahoo.com>

Description A Shiny application, built as a golem package, for exploring World Bank development indicators through interactive country comparison, indicator comparison, population pyramid, ranked bar chart, correlation scatter, and choropleth map views.

License GPL-3

URL https://sedzinfo.github.io/golemworldbank/, https://github.com/sedzinfo/golemworldbank

BugReports https://github.com/sedzinfo/golemworldbank/issues

Encoding UTF-8

LazyData true

LazyDataCompression xz

Roxygen list(markdown = TRUE)

Imports config,
dplyr,
DT,
golem,
htmlwidgets,
plotly,
shiny,
stats,
utils

Config/roxygen2/version 8.1.1

Contents

country_code 2
format_bignum 3

indicator_change . . . 3
indicator_info . . . 4
mfi . . . 5
mfi_population . . . 6
plot_bar . . . 6
plot_cor . . . 7
plot_country_comp . . . 8
plot_indicator_comp . . . 9
plot_map . . . 9
plot_pyramid . . . 10
render_change_table . . . 11
render_index_table . . . 11
run_app . . . 12

Index **13**

`country_code` *Country and region reference table*

Description

Reference metadata mapping each country to its region, income group, and other World Bank classification metadata.

Usage

```
country_code
```

Format

A data frame with 265 rows and 31 columns, including `Country Code`, `Short Name`, `Table Name`, `Long Name`, `Region`, and `Income Group`.

Source

World Bank Open Data (https://data.worldbank.org).

Examples

```
head(country_code)
```

format_bignum *Format a large number with a magnitude suffix*

Description

Abbreviates a numeric value using Tn/Bn/M/K suffixes (trillions, billions, millions, thousands), rounding to the nearest whole unit. Values under 1,000 are returned as-is (coerced to character).

Usage

```
format_bignum(n)
```

Arguments

n	Numeric vector of values to format.

Value

A character vector of formatted values.

Examples

```
format_bignum(c(950, 1500, 2500000, 3200000000, 4100000000000))
#> [1] "950" "2 K" "2 M" "3 Bn" "4 Tn"
```

indicator_change *Indicators that rose or fell between two years for one country*

Description

Compares every indicator a country has a value for in both years and splits them into two tables: the ones that increased from `year1` to `year2` and the ones that decreased, each sorted by percent change (largest change first). Percent change is `(value2 - value1) / |value1| * 100`; it is undefined when the `year1` value is 0, so those indicators are left out of both tables (their names are kept in the `zero_base` element). Indicators that did not change are in neither table.

Usage

```
indicator_change(
  mfi,
  country,
  year1,
  year2,
  include_rates = FALSE,
  include_sign_change = FALSE
)
```

Arguments

mfi	Data frame of World Bank indicator values (long format, with Country Name, Indicator Name, Year, value columns).
country	Character. Country name, as in Country Name.
year1, year2	Numeric or character. The two years to compare.
include_rates	Logical. Keep the rate-of-change indicators (names containing "annual %", "% change" or "growth"); FALSE (default) leaves them out.
include_sign_change	Logical. Keep indicators whose value changed sign between the two years (e.g. net FDI inflows from -8 million to +6.7 billion): their percent change is not meaningful (+82,000%). FALSE (default) leaves them out and lists them in sign_change.

Details

Rates of change ("annual %", "% change", "growth", e.g. "GDP growth (annual %)", "Inflation, consumer prices (annual %)", "... (% change from 1990)") are left out by default: a percent change of a growth rate is not meaningful (a rate going from -0.01 to 1.7 reads as +17,000%). include_rates = TRUE keeps them.

For indicators that are themselves percentages (e.g. "Urban population (% of total population)") the percent change is relative to the old percentage, not a difference in percentage points; the Change column gives the plain difference.

Value

A list with increased and decreased (data frames with columns Indicator, the two years' values, Change and Change %), zero_base (indicators left out because their year1 value is 0) and sign_change (indicators whose value changed sign; left out unless include_sign_change = TRUE).

Examples

```
res <- indicator_change(mfi, country = "Greece", year1 = 2024, year2 = 2025)
head(res$increased)
head(res$decreased)
```

---

indicator_info *Indicator topics and definitions*

Description

World Bank topic and definition of every indicator in mfi, shown in the Index tab.

Usage

```
indicator_info
```

Format

A data frame with 1,497 rows and 3 columns:

Indicator Name World Bank indicator name, as in mfi.

Topic World Bank topic, e.g. "Health: Population: Structure".

Definition Short definition of the indicator.

Source

World Bank Open Data (https://data.worldbank.org), WDISeries.csv.

Examples

```
head(indicator_info)
```

---

mfi *World Bank indicator values*

Description

Long-format World Bank development indicator values, one row per country/indicator/year.

Usage

```
mfi
```

Format

A data frame with 7,478,197 rows and 5 columns:

Country Code ISO country code.

Country Name Country name.

Indicator Name World Bank indicator name.

Year Year of observation (factor).

value Indicator value for that country/year.

Source

World Bank Open Data (https://data.worldbank.org).

Examples

```
head(mfi)
```

mfi_population *Population by age, sex, and country*

Description

Long-format population counts by five-year age group and sex, used to drive the population pyramid module.

Usage

```
mfi_population
```

Format

A data frame with 458,304 rows and 7 columns:

Country Name Country name.

Indicator Name World Bank indicator name.

Year Year of observation (factor).

value Population count.

age Age group.

unit Unit of measurement.

sex "Male" or "Female".

Source

World Bank Open Data (https://data.worldbank.org).

Examples

```
head(mfi_population)
```

plot_bar *Build an animated ranked bar chart*

Description

Plots one indicator's top `top_n` countries per year as a horizontal bar chart animated over `Year` (a "bar chart race"): in every frame the bars are re-ranked, so a country that overtakes another moves up. For a smooth animation the value axis only grows (it follows the largest value seen so far, so bars never jump back when the maximum drops), and a country's missing years between two observed ones are filled by linear interpolation (so it does not drop out of the ranking for a year and come back). Each bar carries the country name and its value; the bar of `country` is red so it can be followed through the race, and the current year is printed large and gray in the bottom right corner. Only positive, complete-case rows are kept. Returns `NULL` when fewer than two rows remain after filtering.

Usage

```
plot_bar(mfi, indicator, font_style, top_n = 20, country = NULL)
```

Arguments

mfi	Data frame of World Bank indicator values (long format, with Country Name, Indicator Name, Year, value columns).
indicator	Character. Indicator name to plot.
font_style	A plotly font list (e.g. list(size = 20, color = "gray25")) applied to the chart's layout.
top_n	Integer. Number of countries shown in each year.
country	Character. Country whose bar is drawn red (NULL for none).

Value

A plotly htmlwidget, or NULL if there is not enough data to plot.

Examples

```
plot_bar(mfi, indicator = "GDP (current US$)", font_style = font_style, country = "Greece")
```

---

plot_cor *Build an animated correlation bubble scatter plot*

Description

Reshapes mfi from long to wide so indicator1 and indicator2 become plottable x/y columns, joins in Region from country_code, and plots a bubble scatter (bubble size = population, color = region), animated over Year, with play/pause controls. Rows with missing values or no region are dropped before plotting.

Usage

```
plot_cor(mfi, country_code, indicator1, indicator2, font_style)
```

Arguments

mfi	Data frame of World Bank indicator values (long format, with Country Name, Indicator Name, Year, value columns). Must also contain a "Population, total" indicator, used for bubble sizing.
country_code	Data frame mapping countries to region, with columns Country Code, Short Name, Region.
indicator1	Character. Indicator name plotted on the x-axis.
indicator2	Character. Indicator name plotted on the y-axis.
font_style	A plotly font list (e.g. list(size = 20, color = "gray25")) applied to the chart's layout.

Value

A `plotly` htmlwidget.

Examples

```
plot_cor(
  mfi,
  country_code,
  indicator1 = "Mortality rate, adult, male (per 1,000 male adults)",
  indicator2 = "Mortality rate, infant (per 1,000 live births)",
  font_style = font_style
)
```

---

plot_country_comp *Build a country comparison line chart*

Description

Plots one indicator's value over time as one line per selected country. Rows with missing values for the given filters are dropped before plotting.

Usage

```
plot_country_comp(mfi, countries, indicator, font_style)
```

Arguments

`mfi`	Data frame of World Bank indicator values (long format, with `Country Name`, `Indicator Name`, `Year`, `value` columns).
`countries`	Character vector of country names to plot.
`indicator`	Character. Indicator name to plot.
`font_style`	A `plotly` font list (e.g. `list(size = 20, color = "gray25")`) applied to the chart's layout.

Value

A `plotly` htmlwidget.

Examples

```
plot_country_comp(
  mfi,
  countries = c("Greece", "Bulgaria"),
  indicator = "Population, total",
  font_style = font_style
)
```

plot_indicator_comp *Build an indicator comparison line chart*

Description

Plots several indicators' values over time, for a single country, as one line per indicator. Rows with missing values for the given filters are dropped before plotting.

Usage

```
plot_indicator_comp(mfi, country, indicators, font_style)
```

Arguments

`mfi`	Data frame of World Bank indicator values (long format, with `Country Name`, `Indicator Name`, `Year`, `value` columns).
`country`	Character. Country name to plot.
`indicators`	Character vector of indicator names to plot.
`font_style`	A `plotly` font list (e.g. `list(size = 20, color = "gray25")`) applied to the chart's layout.

Value

A `plotly` htmlwidget.

Examples

```
plot_indicator_comp(
  mfi,
  country = "Greece",
  indicators = c("Population, female", "Population, male"),
  font_style = font_style
)
```

plot_map *Build an animated choropleth map*

Description

Plots one indicator's value per country as a world choropleth, animated over `Year`. Rows with missing values are dropped before plotting.

Usage

```
plot_map(mfi, indicator, font_style)
```

Arguments

mfi	Data frame of World Bank indicator values (long format, with Country Name, Country Code, Indicator Name, Year, value columns).
indicator	Character. Indicator name to plot.
font_style	A plotly font list (e.g. list(size = 20, color = "gray25")) applied to the chart's layout.

Value

A plotly htmlwidget.

Examples

```
plot_map(mfi, indicator = "Population, total", font_style = font_style)
```

---

plot_pyramid *Build an animated population pyramid*

Description

Plots male (negative, left) and female (positive, right) population counts by age group as horizontal bars, animated over Year. Male values are negated internally so the two sexes render back-to-back, while display_value keeps the original (positive) counts for hover/axis text.

Usage

```
plot_pyramid(mfi_population, country, font_style)
```

Arguments

mfi_population	Data frame of population-by-age/sex values (long format, with Country Name, sex, age, Year, value columns).
country	Character. Country name to plot.
font_style	A plotly font list (e.g. list(size = 20, color = "gray25")) applied to the chart's layout.

Value

A plotly htmlwidget.

Examples

```
plot_pyramid(mfi_population, country = "Greece", font_style = font_style)
```

render_change_table *Render one table of* indicator_change() *as a DataTable*

Description

Values and the absolute change are shown compactly to 3 significant digits (6.02M, 1.42B, 0.0752); sorting still uses the full numbers. The percent change has one decimal, coloured green (increase) or red (decrease).

Usage

```
render_change_table(tab)
```

Arguments

tab	One of the data frames returned by indicator_change() (increased or decreased).

Value

A DT::datatable htmlwidget.

Examples

```
res <- indicator_change(mfi, country = "Greece", year1 = 2000, year2 = 2020)
render_change_table(res$increased)
```

render_index_table *Build the indicator index DataTable*

Description

Builds a non-paginated DT::datatable listing every distinct indicator name present in mfi, with its World Bank topic and definition.

Usage

```
render_index_table(mfi, indicator_info)
```

Arguments

mfi	Data frame of World Bank indicator values, with an Indicator Name column.
indicator_info	Data frame with Indicator Name, Topic and Definition columns (see indicator_info).

Value

A DT::datatable htmlwidget.

Examples

```
render_index_table(mfi, indicator_info)
```

---

run_app *Run the Shiny Application*

Description

Launches the World Bank indicators explorer app.

Usage

```
run_app(
  analytics = FALSE,
  onStart = NULL,
  options = list(),
  enableBookmarking = NULL,
  uiPattern = "/",
  ...
)
```

Arguments

analytics	TRUE to add Google Analytics, through the cookie banner shared with the sites of sedzinfo.github.io, which loads it only after the visitor accepts. Meant for the public deployment (see app.R); off by default, so that running the app yourself sends nothing.
onStart	See shiny::shinyApp().
options	See shiny::shinyApp().
enableBookmarking	See shiny::shinyApp().
uiPattern	See shiny::shinyApp().
...	Arguments passed to golem::with_golem_options().

Examples

```
## Not run:
run_app()

# override any shiny::shinyApp() option, e.g. to pick a fixed port:
run_app(options = list(port = 8080))

## End(Not run)
```

Index

∗ **datasets**
country_code, 2
indicator_info, 4
mfi, 5
mfi_population, 6

country_code, 2

format_bignum, 3

golem::with_golem_options(), *12*

indicator_change, 3
indicator_change(), *11*
indicator_info, 4, *11*

mfi, *4*, 5, *5*
mfi_population, 6

plot_bar, 6
plot_cor, 7
plot_country_comp, 8
plot_indicator_comp, 9
plot_map, 9
plot_pyramid, 10

render_change_table, 11
render_index_table, 11
run_app, 12

shiny::shinyApp(), *12*